

Workshop No. 4 - Containerization, Acceptance, and CI/CD

Software Engineering Seminar

Universidad Distrital Francisco José de Caldas
Systems Engineering Program

Presented by:

Brayan Alejandro Riveros Rodríguez
20201020084

Carlos Andrés Pescador Castro
20182020139

Cristian Santiago Ríos Vásquez
20201020107

Bogotá D.C., Colombia
November 29, 2025

1 Containerization

1.1 Dockerfiles

For this project, each backend includes a custom Dockerfile. The goal of these Dockerfiles is to produce lightweight and efficient container images ready for deployment in local and production environments.

The Java backend Dockerfile uses a multi-stage build. In the first stage, Maven downloads all dependencies and creates the application JAR file. In the second stage, a smaller Amazon Corretto runtime image is used to run the final application. This approach reduces the final image size and improves performance. The container exposes port 8080 and runs the service using a simple `java -jar` command.

Listing 1: Dockerfile for Java Backend

```
FROM maven:3.9.11-amazoncorretto-25-alpine AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests

FROM amazoncorretto:25-alpine AS runtime
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

The Python backend Dockerfile also uses a multi-stage build. The first stage installs dependencies using Poetry and creates a virtual environment inside the project folder. The second stage copies the virtual environment and application code into a clean Python 3.11 image. A non-root user is created for security, and the service is started using Uvicorn on port 8000. A health check is also included to verify that the API is running correctly.

Listing 2: Dockerfile for Python Backend

```
FROM python:3.11-slim AS builder

ENV POETRY_VERSION=1.8.3 \
    POETRY_HOME=/opt/poetry \
    POETRY_NO_INTERACTION=1 \
    POETRY_VIRTUALENVS_IN_PROJECT=1 \
    POETRY_VIRTUALENVS_CREATE=1 \
    POETRY_CACHE_DIR=/tmp/poetry_cache

RUN apt-get update && apt-get install -y --no-install-recommends \
    curl gcc libpq-dev && rm -rf /var/lib/apt/lists/*

RUN curl -sSL https://install.python-poetry.org | python3 -
WORKDIR /app

COPY pyproject.toml poetry.lock* ./
```

```

RUN poetry lock --no-update 2>/dev/null || true && \
poetry install --only main --no-root --no-directory && \
rm -rf $POETRY_CACHE_DIR

FROM python:3.11-slim
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1 \
PATH="/app/.venv/bin:$PATH"

RUN apt-get update && apt-get install -y --no-install-recommends \
libpq5 && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY --from=builder /app/.venv /app/.venv
COPY . .

RUN useradd -m -u 1000 appuser && \
chown -R appuser:appuser /app
USER appuser

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost
:8000/api/v1/health')" || exit 1

CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

These Dockerfiles allow both services to run consistently across different environments and ensure that all dependencies and runtime configurations are properly isolated inside each container.

1.2 Docker Compose Configuration

To orchestrate the application components, we created multiple `docker-compose.yml` and `docker-stack.yml` files for development and deployment. These files define how each container interacts with its database and with the reverse proxy when needed.

For local development, Docker Compose builds the images directly from the Dockerfiles and maps the service ports to the host machine.

For the Java backend, the compose file includes a MySQL database initialized with SQL scripts.

Listing 3: `docker-compose.yml` for Java Backend (Local)

```

version: '3.8'
services:
  java-backend:
    image: java-backend:latest
    build:
      context: .
      dockerfile: Dockerfile
    container_name: java-backend
    ports:

```

```

- "8080:8080"
env_file:
- .env
depends_on:
- java-backend-db
networks:
- java-backend-network

java-backend-db:
image: mysql:8.0
container_name: java-backend-db
env_file:
- .env
volumes:
- ./init_db_scripts/schema_db.sql:/docker-entrypoint-initdb.d/schema_db.sql
- db_data:/var/lib/mysql
healthcheck:
test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "${MYSQL_USER}",
      "-p${MYSQL_PASSWORD}"]
interval: 10s
timeout: 5s
retries: 5
networks:
- java-backend-network

volumes:
db_data:

networks:
java-backend-network:
driver: bridge

```

For the Python backend, the compose file includes a PostgreSQL database with its initialization scripts and health checks.

Listing 4: docker-compose.yml for Python Backend (Local Development)

```

version: '3.8'

services:
python-backend-db:
image: postgres:15-alpine
container_name: python-backend-db
restart: unless-stopped
env_file:
- .env
volumes:
- ./init_db_scripts/schema_db.sql:/docker-entrypoint-initdb.d/schema_db.sql:ro
- postgres_data:/var/lib/postgresql/data
healthcheck:
test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-postgres} -d ${POSTGRES_DB
      :-postgres}"]
interval: 10s
timeout: 5s
retries: 5

```

```

python-backend:
  build:
  context: .
  dockerfile: Dockerfile
  image: python-backend:latest
  container_name: python-backend
  restart: unless-stopped
  ports:
  - "${BACKEND_PORT:-8000}:8000"
  env_file:
  - .env
  depends_on:
  python-backend-db:
  condition: service_healthy
  command: uvicorn src.main:app --host 0.0.0.0 --port 8000 --reload

  volumes:
  postgres_data:
  driver: local

```

Both services load environment variables from .env files and wait for the databases to become healthy before starting.

For production, the docker-stack.yml files define the same services but include additional configuration such as replicas, restart policies, placement constraints, and Traefik labels. These labels allow each backend to be published through HTTPS using different host-names and connected to a shared reverse proxy network.

Listing 5: docker-stack.yml for Java Backend

```

version: '3.8'
services:
  java-backend:
  image: java-backend:latest
  env_file:
  - .env
  depends_on:
  - java-backend-db
  networks:
  - java-backend-network
  - reverse_proxy
  deploy:
  replicas: 1
  restart_policy:
  condition: on-failure
  delay: 10s
  max_attempts: 5
  placement:
  constraints:
  - node.role == manager
  labels:
  - "traefik.enable=true"
  - "traefik.docker.network=reverse_proxy"
  - "traefik.http.routers.java-backend.entrypoints=websecure"
  - "traefik.http.routers.java-backend.tls.certresolver=production"

```

```
- "traefik.http.routers.java-backend.rule=Host('authbackbs.glud.org')"  
- "traefik.http.services.java-backend.loadbalancer.server.port=8080"
```

Listing 6: docker-stack.yml for Python Backend (Docker Swarm)

```
version: '3.8'  
  
services:  
  python-backend:  
    image: python-backend:latest  
    env_file:  
    - .env  
    depends_on:  
    - python-backend-db  
    command: uvicorn src.main:app --host 0.0.0.0 --port 8000  
    networks:  
    - backend-network  
    - reverse_proxy  
    deploy:  
    replicas: 1  
    restart_policy:  
    condition: on-failure  
    placement:  
    constraints:  
    - node.role == manager  
    labels:  
    - "traefik.enable=true"  
    - "traefik.docker.network=reverse_proxy"  
    - "traefik.http.routers.python-backend.entrypoints=websecure"  
    - "traefik.http.routers.python-backend.tls.certresolver=production"  
    - "traefik.http.routers.python-backend.tls=true"  
    - "traefik.http.routers.python-backend.rule=Host('coursefinderbs.glud.org')"  
    - "traefik.http.services.python-backend.loadbalancer.server.port=8000"  
  
  python-backend-db:  
    image: postgres:15-alpine  
    env_file:  
    - .env  
    networks:  
    - backend-network  
    volumes:  
    - ./init_db_scripts/schema_db.sql:/docker-entrypoint-initdb.d/schema_db.sql:ro  
    - postgres_data:/var/lib/postgresql/data  
    healthcheck:  
    test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-postgres}"]  
    interval: 10s  
    timeout: 5s  
    retries: 5  
    deploy:  
    replicas: 1  
    restart_policy:  
    condition: on-failure  
    placement:  
    constraints:  
    - node.role == manager  
  
volumes:
```

```
postgres_data:
  driver: local

networks:
  backend-network:
    driver: overlay
    reverse_proxy:
      external: true
```

Using Docker Compose and Docker Swarm makes it possible to run the system as a group of independent services while keeping the configuration simple, repeatable, and easy to deploy.

1.3 Makefile Usage

The backends also includes a Makefile to simplify common tasks. This file includes commands for installing dependencies, running tests, linting the code, cleaning temporary files, building Docker images, and deploying the stack in Docker Swarm. Those Makefile helps maintain a clean and consistent workflow for all team members.

Listing 7: Makefile for Java Backend

```
install:
  mvn clean package

dev:
  mvn clean spring-boot:run

build:
  docker build -t java-backend:latest .

build-no-cache:
  docker build --no-cache -t java-backend:latest .

compose-up:
  docker compose up -d

compose-up-build:
  docker compose up -d --build

compose-down:
  docker compose down

stack-deploy:
  docker stack deploy -c docker-stack.yml java-backend

stack-remove:
  docker stack rm java-backend

stack-ps:
  docker stack ps java-backend

stack-services:
  docker stack services java-backend
```

Listing 8: Makefile for Python Backend

```
install:
poetry install

install-prod:
poetry install --only main

update:
poetry update

lock:
poetry lock

dev:
poetry run uvicorn src.main:app --reload --host 0.0.0.0 --port 8000

test:
poetry run pytest

test-cov:
poetry run pytest --cov=src --cov-report=html --cov-report=term

lint:
poetry run ruff check src

lint-fix:
poetry run ruff check --fix src

format:
poetry run ruff format src

type-check:
poetry run mypy src

clean:
find . -type d -name "__pycache__" -exec rm -rf {} +
rm -rf htmlcov .coverage

build:
docker build -t python-backend:latest .

build-no-cache:
docker build --no-cache -t python-backend:latest .

compose-up:
docker compose up -d

compose-up-build:
docker compose up -d --build

compose-down:
docker compose down

stack-deploy:
docker stack deploy -c docker-stack.yml python-backend

stack-remove:
docker stack rm python-backend
```



```
stack-ps:
docker stack ps python-backend

stack-services:
docker stack services python-backend
```

1.4 Summary

In summary, the containerization process provides a complete and isolated environment for both backends. The Dockerfiles create optimized and lightweight images, while the Compose and Stack files configure the services, databases, networks, and the reverse proxy for different environments.

In addition to this, both backends include Makefiles that automate common tasks such as installing dependencies, running the application in development mode, building Docker images, and deploying the services with Docker Swarm. These Makefiles help maintain a consistent workflow and make the development and deployment process faster and easier. Overall, these containers make the project simple to run, test, and deploy in both development and production environments.

2 Acceptance Tests with Cucumber

2.1 Overview

For this project, we used Cucumber to create acceptance tests based on the main user stories. The goal of these tests is to verify the system behavior from the point of view of the final user. Cucumber allows us to write tests in a natural language format (Gherkin), making the scenarios easy to understand and simple to maintain.

The acceptance tests help validate that the backends return the correct responses and that the main flows of the system work as expected. Each feature file represents a specific user story, and each scenario describes one possible interaction or outcome.

2.2 Feature Files

The feature files were written using the Gherkin syntax. Each file includes *Given*, *When*, and *Then* steps that describe the expected behavior. The scenarios focus on validating actions such as searching data, creating records, authenticating users, or interacting with the REST API.

Below is a general description of the structure used in the feature files:

- **Given:** Defines the initial state or preconditions. For example, the user is authenticated or the database contains specific records.
- **When:** Describes the action performed by the user, such as sending a request to an API endpoint.

- **Then:** Defines the expected result, such as receiving a correct HTTP status or a JSON response with the expected fields.

This structure helps ensure clarity and consistency across all acceptance tests.

2.3 Step Definitions

For each feature file, we implemented step definitions that connect the Gherkin steps with executable test code. These step definitions send HTTP requests to the backend, receive the responses, and verify the expected results.

The step definitions were organized by functionality. For example:

- Steps for API requests and responses.
- Steps for authentication or user validation.
- Steps for comparing JSON fields or checking HTTP status codes.

This modular structure makes the test suite easier to extend and maintain.

2.4 Execution and Test Results

The Cucumber tests were executed using the project's testing environment. After running the test suite, Cucumber generated a report showing which scenarios passed or failed.

The results showed the following:

- The main user stories behaved correctly.
- The REST API returned the expected responses for valid scenarios.
- Error scenarios were handled correctly, such as missing parameters or invalid requests.

The reports also include execution time and a breakdown of each scenario. These results help confirm that both backends meet the acceptance criteria of the project.

2.5 Summary

The acceptance tests created with Cucumber verify the correct behavior of the application from the user perspective. The feature files, step definitions, and reports provide a clear and structured validation of the system. This ensures that the project satisfies the main user requirements and that the core functionality works as expected.

3 JMeter Test Plans and Results

3.1 Overview

For this project, we used Apache JMeter to perform stress testing on the REST APIs of both backends. The goal of these tests was to evaluate how the system behaves under different levels of load and to identify possible performance problems. JMeter allows us to simulate many users sending requests at the same time, which helps us understand the stability and responsiveness of the application.

3.2 Test Plan Design

The JMeter test plans were created to cover the most important API endpoints. Each plan includes several elements such as:

- **Thread Groups:** Used to simulate multiple virtual users. We configured different numbers of users to test the system under light, medium, and heavy load.
- **HTTP Samplers:** These samplers send requests to the backend endpoints. Each sampler represents a specific API action such as creating, reading, or searching data.
- **Assertions:** Added to verify that each response contains the correct status code and expected fields.
- **Timers:** Used to simulate realistic delays between requests.

This structure ensures that the tests follow an organized and repeatable pattern.

3.3 Scenarios Executed

The following types of scenarios were tested:

- **Single Endpoint Load:** Stress tests for individual endpoints to check their response time and stability.
- **Multiple Endpoint Sequence:** Simulates a user flow with several consecutive operations.
- **High Concurrency Scenario:** Simulates many users working at the same time to evaluate how the system behaves under heavy load.

These scenarios help verify both isolated and combined behavior of the APIs.

3.4 Results and Observations

After executing the test plans, JMeter generated detailed reports. The results showed the following:

- The system maintained stable response times under normal and moderate load.
- No critical errors appeared during concurrent requests.

- Response times increased under heavy load, which is expected in stress testing.
- The backends correctly handled all tested endpoints.

We also reviewed metrics such as response time distribution, throughput, and error rate. These results help understand the system capacity and potential areas for optimization.

3.5 Summary

The JMeter stress tests confirmed that the application can handle different levels of load while keeping stable behavior. The test plans provide a clear view of how the APIs respond in realistic scenarios, and the results help validate that the system is ready for production-like environments.

4 GitHub Actions Workflow

4.1 Overview

For this project, we created a GitHub Actions workflow to automate important tasks such as running tests and building Docker images. The goal of this workflow is to support continuous integration and continuous delivery (CI/CD). Each time we push new code to the repository, the workflow runs automatically and checks that the system continues to work correctly.

4.2 Workflow Structure

The workflow file includes several main steps:

- **Code Checkout:** Downloads the project from the repository so the workflow can work with the files.
- **Environment Setup:** Installs the correct version of Java, Python, or other tools required by the backends.
- **Dependency Installation:** Installs all project dependencies so the tests can run correctly.
- **Test Execution:** Runs the unit tests or acceptance tests defined in the project.
- **Docker Build:** Builds the Docker images for the Java backend, the Python backend, or both.

This structure ensures that each push or pull request is automatically validated.

4.3 Automatic Testing

One important part of the workflow is the automatic test execution. When the workflow runs, it executes the corresponding test suites. If any test fails, the workflow stops and reports the error. This helps identify problems early and ensures code quality.

The results show whether all tests passed and allow developers to correct errors before merging new changes.

4.4 Docker Image Build

The workflow also builds the Docker images for the backends. This guarantees that the Dockerfiles are correct and that the application can be containerized without errors. The images can later be published or used in a deployment environment.

Building the images automatically helps maintain consistency between development and production environments.

4.5 Workflow Results

GitHub Actions provides logs and status messages for each step of the workflow. These messages indicate:

- if the tests passed correctly,
- if the Docker images were built without errors,
- and if the full workflow completed successfully.

These results confirm that the system is stable and that the CI/CD process is working as expected.

4.6 Summary

The GitHub Actions workflow gives the project an automated process for testing and building the application. This CI/CD setup improves code quality, reduces manual work, and ensures that every update to the repository is validated. It also enables faster and more reliable deployments for both backends.

5 Conclusion